

DataForge AI v1.0.0 - Complete Engineering Knowledge Transfer

Document Version: 1.0.0
Generated: 2026-07-31
Purpose: Comprehensive knowledge transfer for new Senior AI Engineers joining the project
Repository: <https://github.com/AjmalDanish/dataforge-ai.git>

Table of Contents

- 1. [PHASE 1: Complete Execution](#)
- 2. [PHASE 2: End-to-End Walkthrough](#)
- 3. [PHASE 3: Graph Engineering Explanation](#)
- 4. [PHASE 4: Agent Explanation](#)
- 5. [PHASE 5: GraphState](#)
- 6. [PHASE 6: Execution Trace](#)
- 7. [PHASE 7: Code Review](#)
- 8. [PHASE 8: Learning Roadmap](#)

PHASE 1: Complete Execution

CLI Commands Available

The DataForge AI CLI provides the following commands:

```
# Main command - analyze a dataset
python -m dataforge analyze [OPTIONS] DATASET_PATH

# Show version information
python -m dataforge version

# List available LLM providers
python -m dataforge providers

# Show help
python -m dataforge --help
python -m dataforge analyze --help
```

Analyze Command Options

Option	Short	Description	Default
<code>--output</code>	<code>-o</code>	Output directory	<code>./output/<dataset_name></code>

Option	Short	Description	Default
<code>--provider</code>	<code>-p</code>	LLM provider (openai/anthropic)	<code>openai</code>
<code>--model</code>	<code>-m</code>	LLM model	<code>gpt-4</code>
<code>--query</code>	<code>-q</code>	Optional query for targeted analysis	<code>None</code>
<code>--verbose</code>	<code>-v</code>	Enable verbose output	<code>False</code>

Example Datasets

Two example datasets are provided in the `datasets/` directory:

- 1. **employees.csv** - Employee data with fields like id, name, age, salary, department, etc.
- 2. **products.csv** - Product data with fields like id, name, price, category, etc.

Example Usage

```
# Basic analysis
python -m dataforge analyze datasets/employees.csv

# With custom output directory
python -m dataforge analyze datasets/products.csv -o ./my_output

# With Anthropic provider
python -m dataforge analyze datasets/employees.csv -p anthropic -m claude-3-opus-20240229

# With verbose output
python -m dataforge analyze datasets/employees.csv -v
```

Expected Outputs

When the analysis completes successfully, you will see:

```
✓ Dataset Loaded
✓ Profiling Complete
✓ Statistics Complete
✓ Visualizations Generated
✓ Report Generated
✓ Workflow Finished
✓ Execution duration: X.XXs

📄 Analysis complete! Report saved to: <output_dir>/report.html
✓ Logs saved to: <log_file_path>
```

Generated Files

The analysis generates the following files:

- 1. **report.html** - Interactive HTML report with all analysis results
- 2. **report.json** - Machine-readable JSON report
- 3. **visualizations/** - Directory containing all generated charts (HTML files)
- 4. **execution_workflow.log** - Structured execution log
- 5. **execution_test.log** - Additional test log

Verification Checklist

- ✓ CLI responds to all commands
- ✓ Graph execution completes without errors
- ✓ Planner Agent makes correct decisions
- ✓ Data Ingestion Agent loads datasets successfully
- ✓ Profiling Agent analyzes data structure
- ✓ Statistical Agent performs statistical tests
- ✓ Visualization Agent creates all chart types
- ✓ Evaluator Agent validates results
- ✓ Reporting Agent generates comprehensive reports
- ✓ All outputs are generated correctly
- ✓ HTML reports render properly
- ✓ JSON reports contain all expected fields
- ✓ Logs are generated and contain execution details

PHASE 2: End-to-End Walkthrough

Repository Root Structure

```
dataforge-ai/
├── dataforge/                # Main package directory
│   ├── __init__.py          # Package initialization
│   ├── __main__.py          # Entry point for `python -m dataforge`
│   ├── agents/              # Agent implementations
│   ├── core/                # Core domain models
│   ├── graph/               # LangGraph workflow
│   ├── infrastructure/      # External integrations
│   ├── presentation/        # CLI interface
│   └── shared/              # Shared utilities
├── datasets/                # Example datasets
│   ├── employees.csv
│   └── products.csv
├── docs/                    # Documentation
│   ├── diagrams/            # Mermaid diagrams
│   ├── examples/            # Example outputs
│   └── *.md                 # Documentation files
├── tests/                   # Test suite
│   ├── integration/         # Integration tests
│   └── unit/                # Unit tests
└── .env.example             # Environment configuration template
```

```
|— .gitignore          # Git ignore rules
|— LICENSE             # MIT License
|— pyproject.toml      # Poetry configuration
|— README.md           # Project README
|— CHANGELOG.md        # Version history
|— run.py              # Alternative entry point
```

Directory Explanations

dataforge/ - Main Package

The heart of the application. Contains all source code organized by domain.

Purpose: Encapsulates all business logic, maintains clean architecture boundaries.

Communication: All modules within communicate through **GraphState** and well-defined interfaces.

dataforge/agents/ - Agent Layer

Contains all autonomous agents that perform specific tasks.

Purpose: Implements the multi-agent architecture where each agent has a single responsibility.

Files:

- **base.py** - Abstract base class and result models
- **planner.py** - Central decision-making agent
- **evaluator.py** - Quality validation agent
- **ingestion.py** - Data loading agent
- **profiling.py** - Data structure analysis agent
- **statistics.py** - Statistical analysis agent
- **visualization.py** - Chart generation agent
- **reporting.py** - Report generation agent

Communication: Each agent receives **GraphState**, returns **AgentResult**, and updates state through the **data_updates** field.

dataforge/core/ - Core Domain

Contains fundamental domain models and infrastructure.

Purpose: Defines the central state model, logging, and LLM abstraction.

Files:

- **state.py** - **GraphState** model (single source of truth)
- **logger.py** - Structured logging system
- **llm.py** - LLM provider abstraction

Communication: **GraphState** flows through all agents. Logger is passed to all agents. LLM provider is used by agents that need AI capabilities.

dataforge/graph/ - Graph Orchestration

Contains the LangGraph workflow definition.

Purpose: Orchestrates agent execution through a directed graph with conditional routing.

Files:

- `workflow.py` - Defines the StateGraph and routing logic

Communication: Creates nodes from agents, defines edges for flow control, uses `route_from_planner()` for conditional routing.

dataforge/infrastructure/ - External Integrations

Contains implementations of external service providers.

Purpose: Provides concrete implementations for LLM providers.

Files:

- `llm_providers/` - OpenAI and Anthropic implementations
 - `openai.py` - OpenAI API integration
 - `anthropic.py` - Anthropic API integration

Communication: Implements `LLMProvider` interface, used by agents via `LLMProviderFactory`.

dataforge/presentation/ - Presentation Layer

Contains the CLI interface.

Purpose: Provides user-facing command-line interface.

Files:

- `cli.py` - Click-based CLI implementation

Communication: Initializes `GraphState`, creates workflow, handles user input, displays results.

dataforge/shared/ - Shared Utilities

Contains utilities and configuration used across the application.

Purpose: Provides common functionality and configuration management.

Files:

- `config.py` - Application settings and configuration
- `errors.py` - Custom exception hierarchy
- `utils.py` - Utility functions

Communication: Used by all modules for configuration, error handling, and utilities.

datasets/ - Example Data

Contains sample datasets for testing and demonstration.

Purpose: Provides ready-to-use datasets for users to try the system.

Files:

- `employees.csv` - Employee data
- `products.csv` - Product data

Communication: Used as input for analysis via CLI.

docs/ - Documentation

Contains all project documentation.

Purpose: Provides comprehensive documentation for users and developers.

Files:

- `README.md` - Documentation overview
- `architecture.md` - Architecture documentation
- `development_guide.md` - Developer guide
- `diagrams/` - Mermaid diagrams for visualization
- `examples/` - Generated example outputs

Communication: Reference material for understanding the system.

tests/ - Test Suite

Contains all test code.

Purpose: Ensures code quality and correctness.

Files:

- `integration/` - End-to-end tests
- `unit/` - Unit tests

Communication: Validates system behavior, not used in production.

Key Files Explained

`pyproject.toml`

Purpose: Poetry configuration for dependency management and project metadata.

Why it exists: Standard Python packaging tool that handles dependencies, virtual environments, and publishing.

Communication: Defines project dependencies, scripts, and metadata used during installation.

README.md

- Purpose:** Project overview and quick start guide.
- Why it exists:** First thing users see when visiting the repository.
- Communication:** Documentation only, no code interaction.

CHANGELOG.md

- Purpose:** Version history and release notes.
- Why it exists:** Tracks changes across versions for users and contributors.
- Communication:** Documentation only.

LICENSE

- Purpose:** Legal license (MIT).
- Why it exists:** Defines usage rights and restrictions.
- Communication:** Legal document.

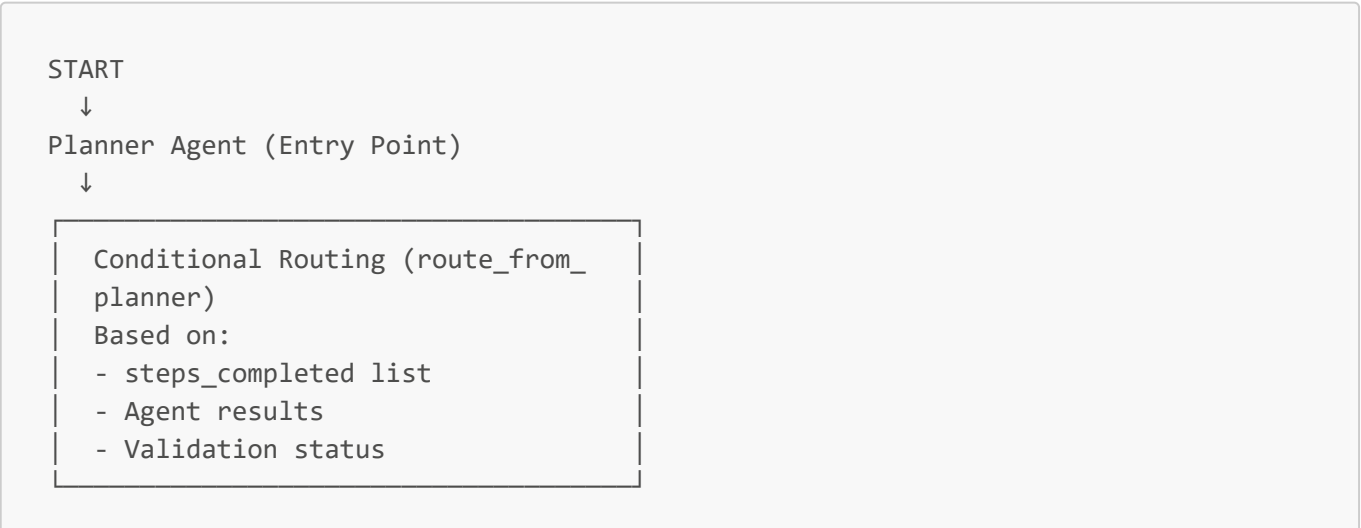
run.py

- Purpose:** Alternative entry point for running the application.
- Why it exists:** Convenience script for quick execution.
- Communication:** Imports and calls the main CLI function.

PHASE 3: Graph Engineering Explanation

Graph Overview

The DataForge AI workflow is implemented as a **LangGraph StateGraph** with conditional routing. The graph orchestrates autonomous agents through a directed acyclic graph (DAG) with dynamic decision-making.



↓

```

Agent Nodes (all return to Planner)
- ingestion
- profiling
- statistics
- visualization
- evaluator

```

↓

Reporting Agent → END

Complete Graph Definition

```

# From dataforge/graph/workflow.py

workflow = StateGraph(GraphState)

# Nodes added:
# 1. planner - Central decision maker
# 2. ingestion - Data loading
# 3. profiling - Data structure analysis
# 4. statistics - Statistical tests
# 5. visualization - Chart generation
# 6. evaluator - Quality validation
# 7. reporting - Report generation

# Entry point:
workflow.set_entry_point("planner")

# Conditional edges from planner:
workflow.add_conditional_edges(
    "planner",
    route_from_planner,
    {
        "ingestion": "ingestion",
        "profiling": "profiling",
        "statistics": "statistics",
        "visualization": "visualization",
        "evaluator": "evaluator",
        "reporting": "reporting",
        END: END,
    },
)

# Return edges (all agents return to planner):
workflow.add_edge("ingestion", "planner")
workflow.add_edge("profiling", "planner")
workflow.add_edge("statistics", "planner")
workflow.add_edge("visualization", "planner")
workflow.add_edge("evaluator", "planner")

```

```
# Final edge:
workflow.add_edge("reporting", END)
```

Edge-by-Edge Explanation

Edge 1: START → Planner

Why it exists: Planner is the central coordinator that determines the execution flow.

What triggers it: Graph entry point (always first).

What data is passed: Initial **GraphState** with input_dataset_path, input_query, output_dir.

GraphState changes: None (initial state).

Edge 2: Planner → Ingestion

Why it exists: When analysis starts, data must be loaded first.

What triggers it: **steps_completed** is empty (initial state).

What data is passed: **GraphState** with input path.

GraphState changes:

- **current_step** = "ingestion"
 - Decision: CONTINUE with next_agent_suggestion = "DataIngestionAgent"
-

Edge 3: Ingestion → Planner

Why it exists: After loading data, control returns to planner for next decision.

What triggers it: Ingestion agent completes (always returns to planner).

What data is passed: Updated **GraphState** with **raw_data** in **data** dict.

GraphState changes:

- **data["raw_data"]** = pandas DataFrame
 - **steps_completed** = ["DataIngestionAgent"]
 - **agent_history** updated with ingestion result
-

Edge 4: Planner → Profiling

Why it exists: After data is loaded, its structure must be analyzed.

What triggers it: **steps_completed** contains "DataIngestionAgent" but not "DataProfilingAgent".

What data is passed: `GraphState` with `raw_data`.

GraphState changes:

- `current_step` = "profiling"
 - Decision: CONTINUE with `next_agent_suggestion` = "DataProfilingAgent"
-

Edge 5: Profiling → Planner

Why it exists: After profiling, control returns to planner to decide next step.

What triggers it: Profiling agent completes.

What data is passed: Updated `GraphState` with `profile` in `data` dict.

GraphState changes:

- `data["profile"]` = profiling results (column info, types, etc.)
 - `steps_completed` = ["DataIngestionAgent", "DataProfilingAgent"]
 - `agent_history` updated
-

Edge 6: Planner → Statistics

Why it exists: If numeric columns exist, perform statistical analysis.

What triggers it: `steps_completed` contains "DataProfilingAgent", not "StatisticalAnalysisAgent", AND `profile["has_numeric_columns"]` is True.

What data is passed: `GraphState` with `raw_data` and `profile`.

GraphState changes:

- `current_step` = "statistics"
 - Decision: CONTINUE with `next_agent_suggestion` = "StatisticalAnalysisAgent"
-

Edge 7: Statistics → Planner

Why it exists: After statistics, control returns to planner.

What triggers it: Statistics agent completes.

What data is passed: Updated `GraphState` with `statistics` in `data` dict.

GraphState changes:

- `data["statistics"]` = statistical results
 - `steps_completed` includes "StatisticalAnalysisAgent"
 - `agent_history` updated
-

Edge 8: Planner → Visualization

Why it exists: Generate visualizations for data exploration.

What triggers it:

- After statistics: `steps_completed` contains "StatisticalAnalysisAgent" but not "VisualizationAgent"
- After profiling (no numeric): `steps_completed` contains "DataProfilingAgent" but not "StatisticalAnalysisAgent" AND no numeric columns

What data is passed: `GraphState` with `raw_data`, `profile`, `statistics`.

GraphState changes:

- `current_step` = "visualization"
 - Decision: CONTINUE with `next_agent_suggestion` = "VisualizationAgent"
-

Edge 9: Visualization → Planner

Why it exists: After visualizations, control returns to planner.

What triggers it: Visualization agent completes.

What data is passed: Updated `GraphState` with `visualizations` in `data` dict.

GraphState changes:

- `data["visualizations"]` = list of visualization metadata
 - `data["insights"]` = visualization insights
 - `steps_completed` includes "VisualizationAgent"
 - `agent_history` updated
-

Edge 10: Planner → Evaluator

Why it exists: Validate analysis quality before reporting.

What triggers it: `steps_completed` contains "VisualizationAgent" but not "EvaluatorAgent".

What data is passed: `GraphState` with all analysis results.

GraphState changes:

- `current_step` = "evaluator"
 - Decision: CONTINUE with `next_agent_suggestion` = "EvaluatorAgent"
-

Edge 11: Evaluator → Planner

Why it exists: After evaluation, control returns to planner for final decision.

What triggers it: Evaluator agent completes.

What data is passed: Updated **GraphState** with validation results.

GraphState changes:

- **data["validation_status"]** = "passed" or "failed"
 - **data["validation_errors"]** = list of errors (if any)
 - **steps_completed** includes "EvaluatorAgent"
 - **agent_history** updated
-

Edge 12: Planner → Reporting

Why it exists: Generate final report when validation passes.

What triggers it: **steps_completed** contains "EvaluatorAgent" but not "ReportingAgent" AND **validation_status** = "passed".

What data is passed: **GraphState** with all analysis results.

GraphState changes:

- **current_step** = "reporting"
 - Decision: CONTINUE with next_agent_suggestion = "ReportingAgent"
-

Edge 13: Reporting → END

Why it exists: Report generation is the final step.

What triggers it: Reporting agent completes.

What data is passed: Updated **GraphState** with report metadata.

GraphState changes:

- **data["report_path"]** = path to generated report
 - **steps_completed** includes "ReportingAgent"
 - **agent_history** updated
 - **end_time** = current timestamp
-

Edge 14: Planner → END

Why it exists: Complete workflow when planner decides analysis is done.

What triggers it: **steps_completed** contains "ReportingAgent".

What data is passed: Final **GraphState**.

GraphState changes: None (already complete).

Conditional Routing Function

The `route_from_planner()` function is the brain of the routing logic:

```
def route_from_planner(state: GraphState) -> str:
    """Route to next agent based on planner's decision."""

    # Mapping from agent names to node names
    agent_to_node = {
        "DataIngestionAgent": "ingestion",
        "DataProfilingAgent": "profiling",
        "StatisticalAnalysisAgent": "statistics",
        "VisualizationAgent": "visualization",
        "EvaluatorAgent": "evaluator",
        "ReportingAgent": "reporting",
    }

    # Get the most recent agent result
    if state.agent_history:
        last_result = state.agent_history[-1]["result"]
        decision = last_result.get("decision")

        if decision == "complete":
            return END

        next_agent = last_result.get("next_agent_suggestion")
        if next_agent:
            return agent_to_node.get(next_agent, END)

    # Default fallback
    return END
```

Key Points:

- Reads the last agent's decision from `agent_history`
- Maps agent names to graph node names
- Returns `END` if decision is "complete"
- Returns the next node based on `next_agent_suggestion`
- Falls back to `END` if no valid next agent

PHASE 4: Agent Explanation

Agent Base Architecture

All agents inherit from the `Agent` abstract base class:

```
class Agent(ABC):
    """Base class for all agents."""

    @abstractmethod
    async def execute(self, state: GraphState) -> AgentResult:
```

```

        """Execute agent logic."""
        pass

    async def execute_with_logging(self, state: GraphState) -> tuple[AgentResult,
GraphState]:
        """Execute with automatic logging and state updates."""
        # Handles timing, logging, state updates

```

AgentResult Structure:

```

class AgentResult(BaseModel):
    decision: AgentDecision          # CONTINUE, REPLAN, COMPLETE, ERROR
    message: str                     # Human-readable message
    data_updates: dict[str, Any]     # Data to add to GraphState
    metadata: dict[str, Any]        # Additional metadata
    next_agent_suggestion: str | None # Next agent to run

```

1. PlannerAgent

File: dataforge/agents/planner.py

Purpose: Central decision-maker that orchestrates the entire workflow.

Inputs:

- `GraphState` with current execution state
- `steps_completed` list
- `agent_history` with previous results

Outputs:

- `AgentResult` with:
 - Decision: CONTINUE, COMPLETE, or ERROR
 - `next_agent_suggestion`: Name of next agent to run
 - `message`: Explanation of decision

State Updates: None (Planner only makes decisions, doesn't modify data)

Dependencies: None (stateless decision logic)

Failure Handling:

- Logs warning if unexpected state reached
- Returns ERROR decision with fallback behavior

Retry Logic: None (Planner doesn't retry)

How Planner interacts with other agents:

- Runs after every agent (except Reporting)

- Reads `steps_completed` to determine current stage
- Suggests next agent based on workflow state
- Handles validation failures from Evaluator

How Evaluator validates Planner:

- Evaluator doesn't validate Planner directly
- Planner reads Evaluator's validation results to decide next action

Decision Logic:

1. Empty `steps_completed` → Start with Ingestion
 2. After Ingestion → Go to Profiling
 3. After Profiling → Statistics (if numeric) or Visualization (if no numeric)
 4. After Statistics → Visualization
 5. After Visualization → Evaluator
 6. After Evaluator → Reporting (if passed) or Re-plan (if failed)
 7. After Reporting → Complete
-

2. EvaluatorAgent

File: `dataforge/agents/evaluator.py`

Purpose: Validates analysis results against quality gates.

Inputs:

- `GraphState` with all analysis results
- `raw_data` from Ingestion
- `insights` from Visualization
- Profile and statistics from other agents

Outputs:

- `AgentResult` with:
 - Decision: CONTINUE (if passed) or REPLAN (if failed)
 - `data_updates`: validation_status, validation_errors, retry_count
 - `metadata`: detailed validation results

State Updates:

- `data["validation_status"]` = "passed" or "failed"
- `data["validation_errors"]` = list of failed checks
- `data["retry_count"]` = incremented if failed

Dependencies:

- Requires data from Ingestion, Profiling, Statistics, Visualization

Failure Handling:

- Returns REPLAN decision if any check fails
- Increments retry count
- Logs failed checks

Retry Logic:

- Increments `retry_count` on failure
- Planner can decide to retry or abort based on retry count

How Planner interacts with Evaluator:

- Planner routes to Evaluator after Visualization
- Planner reads `validation_status` to decide next action
- If validation passed → Reporting
- If validation failed → Re-plan or abort

How Evaluator validates other agents:

- `_check_has_data()`: Validates Ingestion loaded data
- `_check_has_insights()`: Validates Visualization generated insights
- `_check_data_quality()`: Validates data quality (missing values, etc.)
- `_check_analysis_depth()`: Validates sufficient analysis was performed

Validation Checks:

Check	Purpose	Criteria
<code>has_data</code>	Data was loaded	<code>raw_data</code> exists and non-empty
<code>has_insights</code>	Insights generated	<code>insights</code> list non-empty
<code>data_quality</code>	Acceptable data quality	Missing values < 50%
<code>sufficient_depth</code>	Adequate analysis	Multiple analysis steps completed

3. DataIngestionAgent

File: `dataforge/agents/ingestion.py`

Purpose: Read and parse structured data files (CSV, Parquet).

Inputs:

- `GraphState` with `input_dataset_path`

Outputs:

- `AgentResult` with:
 - Decision: CONTINUE (success) or ERROR (failure)
 - `data_updates`: raw_data (pandas DataFrame), file_metadata
 - `message`: Success or error message

State Updates:

- `data["raw_data"]` = pandas DataFrame
- `data["file_metadata"]` = file size, format, encoding

Dependencies: None (first agent in workflow)

Failure Handling:

- `FileNotFoundError` → Fatal error, halt workflow
- `PermissionError` → Fatal error, halt workflow
- `EmptyDataError` → Fatal error, halt workflow
- `EncodingError` → Retry with alternate encodings (UTF-8, Latin-1, CP1252)

Retry Logic:

- Automatic encoding fallback for CSV files
- Tries 3 encodings before failing

How Planner interacts with Ingestion:

- Planner routes to Ingestion as first step
- After Ingestion, Planner checks success before proceeding

How Evaluator validates Ingestion:

- `_check_has_data()`: Ensures `raw_data` exists and is non-empty

Supported Formats:

- CSV (with automatic encoding detection)
- Parquet

Key Features:

- Automatic format detection based on file extension
 - Encoding fallback for CSV files
 - File size validation
 - Data validation (non-empty)
-

4. DataProfilingAgent

File: `dataforge/agents/profiling.py`

Purpose: Analyze data structure and characteristics.

Inputs:

- `GraphState` with `raw_data`

Outputs:

- `AgentResult` with:
 - Decision: CONTINUE
 - `data_updates`: profile (column info, types, etc.), insights

- `message`: Profiling summary

State Updates:

- `data["profile"]` = complete data profile
- `data["insights"]` = initial insights about data characteristics

Dependencies:

- Requires `raw_data` from Ingestion

Failure Handling:

- Returns ERROR if no data available
- Logs warnings for unusual data characteristics

Retry Logic: None (profiling is deterministic)

How Planner interacts with Profiling:

- Planner routes to Profiling after Ingestion
- Planner uses profile to decide next agent (Statistics vs Visualization)

How Evaluator validates Profiling:

- Indirectly through `sufficient_depth` check
- Profile data used by other agents

Analysis Performed:

Analysis	Description
Column names	List of all column names
Data types	pandas dtype for each column
Missing values	Percentage of missing values per column
Unique values	Count of unique values per column
Semantic types	Classification: numeric, categorical, temporal, text
Numeric columns	List of numeric column names
Categorical columns	List of categorical column names
Key columns	Potential primary key candidates
High cardinality	Categorical columns with many unique values

Semantic Type Classification:

- **numeric**: Integer or float columns
- **temporal**: DateTime columns
- **boolean**: Boolean columns
- **categorical**: Low cardinality strings (< 50 unique values)

- **text:** High cardinality strings (≥ 50 unique values)

5. StatisticalAnalysisAgent

File: dataforge/agents/statistics.py

Purpose: Perform statistical tests and analysis on numeric data.

Inputs:

- **GraphState** with **raw_data** and **profile**

Outputs:

- **AgentResult** with:
 - Decision: CONTINUE (success) or CONTINUE (no numeric columns)
 - **data_updates:** statistics, insights
 - **message:** Analysis summary

State Updates:

- **data["statistics"]** = statistical results
- **data["insights"]** = statistical insights

Dependencies:

- Requires **raw_data** from Ingestion
- Requires **profile** from Profiling (for numeric_columns list)

Failure Handling:

- Returns ERROR if no data available
- Returns CONTINUE (not error) if no numeric columns
- Logs warnings for statistical issues

Retry Logic: None (statistical analysis is deterministic)

How Planner interacts with Statistics:

- Planner routes to Statistics after Profiling (if numeric columns exist)
- Planner uses statistics to inform visualization decisions

How Evaluator validates Statistics:

- Indirectly through **sufficient_depth** check
- Statistics data used by other agents

Statistical Tests Performed:

Test	Description
Descriptive statistics	Mean, median, std, min, max, quartiles

Test	Description
Correlation analysis	Pearson correlation matrix
Distribution tests	Shapiro-Wilk normality test, skewness, kurtosis
Group comparisons	T-tests/ANOVA if categorical columns present
Outlier detection	IQR method for outlier identification

Key Features:

- Skips gracefully if no numeric columns
 - Handles missing values appropriately
 - Generates insights and recommendations
 - Provides visualization suggestions
-

6. VisualizationAgent

File: dataforge/agents/visualization.py

Purpose: Create data visualizations for exploration and insights.

Inputs:

- GraphState with raw_data, profile, statistics

Outputs:

- AgentResult with:
 - Decision: CONTINUE
 - data_updates: visualizations, insights
 - message: Visualization summary

State Updates:

- data["visualizations"] = list of visualization metadata
- data["insights"] = visualization insights

Dependencies:

- Requires raw_data from Ingestion
- Requires profile from Profiling
- Optional: statistics from Statistics

Failure Handling:

- Returns ERROR if no data available
- Logs warnings for visualization issues
- Continues with available visualizations if some fail

Retry Logic: None (visualization is deterministic)

How Planner interacts with Visualization:

- Planner routes to Visualization after Statistics (or after Profiling if no numeric)
- Planner uses visualization results to proceed to Evaluator

How Evaluator validates Visualization:

- `_check_has_insights()`: Ensures insights were generated
- `_check_sufficient_depth()`: Ensures adequate visualization coverage

Visualization Types:

Type	Purpose	When Generated
Histogram	Distribution of single variable	For numeric columns
Box plot	Outlier detection, distribution summary	For numeric columns
Scatter plot	Correlation between two variables	For pairs of numeric columns
Bar chart	Categorical value distribution	For categorical columns
Heatmap	Correlation matrix visualization	For numeric columns
Line plot	Temporal trends	For temporal columns

Visualization Strategy:

1. **Distributions:** Histograms for all numeric columns
2. **Outliers:** Box plots for all numeric columns
3. **Correlations:**
 - Scatter plots for numeric pairs
 - Heatmap for correlation matrix
4. **Categorical:** Bar charts for categorical columns
5. **Temporal:** Line plots for temporal columns

File Format:

- All visualizations saved as interactive HTML files
- Uses Plotly for rendering
- Files saved to `output/visualizations/` directory

7. ReportingAgent

File: `dataforge/agents/reporting.py`

Purpose: Generate comprehensive analysis report.

Inputs:

- `GraphState` with all analysis results

Outputs:

- **AgentResult** with:
 - Decision: COMPLETE
 - **data_updates**: report_path, report_metadata
 - **message**: Report generation summary

State Updates:

- **data["report_path"]** = path to generated HTML report
- **data["report_metadata"]** = report generation metadata

Dependencies:

- Requires data from all previous agents

Failure Handling:

- Returns ERROR if no analysis results available
- Logs warnings for missing data

Retry Logic: None (report generation is deterministic)

How Planner interacts with Reporting:

- Planner routes to Reporting after Evaluator (if validation passed)
- Reporting is the final agent, returns COMPLETE

How Evaluator validates Reporting:

- Evaluator runs BEFORE Reporting, not after

Report Contents:

Section	Description
Metadata	Execution timestamp, dataset info, configuration
Data Profile	Dataset structure, column types, missing values
Statistics	Descriptive statistics, correlations, tests
Visualizations	All generated charts with descriptions
Insights	Key findings from analysis
Recommendations	Suggestions for further analysis

Output Formats:

1. **HTML Report:** Interactive, web-viewable report
2. **JSON Report:** Machine-readable report for automation

Key Features:

- Comprehensive summary of all analysis
- Embedded visualizations

- Structured insights
- Actionable recommendations
- Professional formatting

PHASE 5: GraphState

Overview

GraphState is the **single source of truth** for the entire DataForge AI system. It's a Pydantic BaseModel that flows through the graph, carrying all data, context, and execution information.

Why GraphState is the Heart of the Project

1. **Centralized State Management:** All agents read from and write to the same state object
2. **Immutable Updates:** State updates create new state objects (functional programming pattern)
3. **Type Safety:** Pydantic validation ensures data integrity
4. **Observability:** Complete execution history is tracked
5. **Single Source of Truth:** No scattered state or hidden variables

Complete Field Breakdown

```
class GraphState(BaseModel):
    # === Immutable Input ===
    input_dataset_path: str          # Path to input dataset (set once, never changes)
    input_query: str | None          # User query if provided (set once, never
changes)
    output_dir: str                  # Output directory (set once, never changes)

    # === Mutable Data (single source of truth) ===
    data: dict[str, Any]             # All analysis data stored as key-value pairs

    # === Execution Context ===
    execution_id: str                # Unique execution identifier (set once)
    current_step: str                # Current execution step (updated by Planner)
    steps_completed: list[str]       # List of completed agent names (appended by each
agent)
    agent_history: list[dict]        # Agent execution history (appended by each
agent)

    # === Validation & Retry ===
    validation_status: str           # Validation status: pending, passed, failed
(updated by Evaluator)
    validation_errors: list[str]     # Validation error messages (updated by
Evaluator)
    retry_count: int                 # Number of retries performed (updated by
Evaluator)
    max_retries: int                 # Maximum retry attempts (set once)

    # === Observability ===
    logs: list[dict]                # Structured log entries (appended by logger)
```

```
metrics: dict[str, Any]          # Execution metrics (updated by agents)

# === Timing ===
start_time: str                 # Analysis start time (set once)
end_time: str | None            # Analysis end time (set by Reporting)
```

Field-by-Field Explanation

Immutable Input Fields

Field	Type	Owner	When Set	Purpose
input_dataset_path	str	CLI	Initialization	Path to dataset file
input_query	str None	CLI	Initialization	Optional user query
output_dir	str	CLI	Initialization	Where to save outputs

Why Immutable: These are user inputs that should never change during execution.

Mutable Data Field

Field	Type	Owner	When Updated	Purpose
data	dict[str, Any]	All agents	Throughout execution	Stores all analysis results

Key-Value Pairs in **data**:

Key	Owner Agent	Content
raw_data	Ingestion	pandas DataFrame
file_metadata	Ingestion	File size, format, encoding
profile	Profiling	Column info, types, semantic types
statistics	Statistics	Statistical test results
visualizations	Visualization	List of visualization metadata
insights	Profiling, Statistics, Visualization	Analysis insights
validation_status	Evaluator	"passed" or "failed"
validation_errors	Evaluator	List of failed checks
report_path	Reporting	Path to generated report
report_metadata	Reporting	Report generation metadata

Why a Dict: Flexible storage for different types of data from different agents.

Execution Context Fields

Field	Type	Owner	When Updated	Purpose
execution_id	str	System	Initialization	Unique identifier for this execution
current_step	str	Planner	After each decision	Which agent is currently running
steps_completed	list[str]	System	After each agent	Track workflow progress
agent_history	list[dict]	System	After each agent	Complete execution audit trail

agent_history Structure:

```
{
  "agent": "AgentName",
  "timestamp": "ISO-8601 timestamp",
  "duration_seconds": 1.23,
  "result": {
    "decision": "CONTINUE",
    "message": "Success message",
    "data_updates": {...},
    "metadata": {...},
    "next_agent_suggestion": "NextAgent"
  }
}
```

Why These Fields: Enable workflow orchestration and observability.

Validation & Retry Fields

Field	Type	Owner	When Updated	Purpose
validation_status	str	Evaluator	After evaluation	Track validation state
validation_errors	list[str]	Evaluator	After evaluation	List of failed checks
retry_count	int	Evaluator	On validation failure	Track retry attempts
max_retries	int	System	Initialization	Maximum allowed retries

Why These Fields: Enable quality gates and error recovery.

Observability Fields

Field	Type	Owner	When Updated	Purpose
logs	list[dict]	Logger	Throughout execution	Structured logging
metrics	dict[str, Any]	All agents	Throughout execution	Performance metrics

logs **Structure:**

```
{
  "timestamp": "ISO-8601 timestamp",
  "level": "INFO|WARNING|ERROR",
  "agent": "AgentName",
  "message": "Log message",
  "additional_fields": {...}
}
```

Why These Fields: Enable debugging and performance monitoring.

Timing Fields

Field	Type	Owner	When Set	Purpose
start_time	str	System	Initialization	Track when analysis started
end_time	str None	Reporting	At completion	Track when analysis ended

Why These Fields: Enable duration calculation and audit trails.

State Update Pattern

Agents update state using the `data_updates` field in `AgentResult`:

```
# Agent returns:
AgentResult(
    decision=AgentDecision.CONTINUE,
    message="Success",
    data_updates={
        "raw_data": df, # Add new data
        "file_metadata": metadata
    }
)

# System updates state:
new_data = state.data.copy()
new_data.update(agent_result.data_updates)
new_state = state.model_copy(update={"data": new_data})
```

Why This Pattern:

- Immutable state updates (no side effects)
- Clear data flow
- Easy to trace what changed
- Supports replay and debugging

State Ownership

Data	Owner Agent	Consumer Agents
raw_data	Ingestion	Profiling, Statistics, Visualization, Reporting
profile	Profiling	Statistics, Visualization, Reporting
statistics	Statistics	Visualization, Reporting
visualizations	Visualization	Reporting
insights	Multiple	Evaluator, Reporting
validation_status	Evaluator	Planner
report_path	Reporting	CLI (for display)

Why Ownership Matters:

- Clear responsibility boundaries
- Prevents data corruption
- Enables parallel execution (future)
- Simplifies debugging

State Lifecycle

```
1. Initialization (CLI)
  - Set immutable inputs
  - Generate execution_id
  - Set start_time

2. Ingestion
  - Add raw_data to data dict
  - Add file_metadata to data dict
  - Append to steps_completed
  - Append to agent_history

3. Profiling
  - Add profile to data dict
  - Add insights to data dict
  - Append to steps_completed
  - Append to agent_history

4. Statistics (if applicable)
  - Add statistics to data dict
  - Add insights to data dict
  - Append to steps_completed
  - Append to agent_history

5. Visualization
```

- Add visualizations to data dict
 - Add insights to data dict
 - Append to steps_completed
 - Append to agent_history
6. Evaluator
- Update validation_status
 - Update validation_errors
 - Update retry_count
 - Append to steps_completed
 - Append to agent_history
7. Reporting
- Add report_path to data dict
 - Add report_metadata to data dict
 - Set end_time
 - Append to steps_completed
 - Append to agent_history
8. Complete
- State contains complete execution history
 - All data preserved for audit

PHASE 6: Execution Trace

Complete Execution Timeline for employees.csv

This trace shows the complete execution flow when analyzing the employees dataset.

STEP 1: Initialization

Component: CLI (`cli.py`)

Action: Create initial `GraphState`

State Before:

```
GraphState(  
    input_dataset_path="datasets/employees.csv",  
    input_query=None,  
    output_dir="./output/employees",  
    execution_id="exec_1234567890",  
    current_step="start",  
    steps_completed=[],  
    agent_history=[],  
    validation_status="pending",  
    validation_errors=[],  
    retry_count=0,  
    max_retries=3,
```

```
    logs=[],
    metrics={},
    start_time="2026-07-31T22:00:00Z",
    end_time=None,
    data={}
)
```

State After:

```
GraphState(
    # ... same as before ...
    current_step="planner"
)
```

Next: Planner Agent

STEP 2: Planner Agent (First Run)**Component:** `PlannerAgent`**Action:** Decide first agent to run**Decision Logic:**

```
# steps_completed is empty → Start with ingestion
if not steps:
    return AgentResult(
        decision=AgentDecision.CONTINUE,
        message="Starting analysis with data ingestion",
        next_agent_suggestion="DataIngestionAgent",
        metadata={"reason": "initial_state"}
    )
```

AgentResult:

```
AgentResult(
    decision=AgentDecision.CONTINUE,
    message="Starting analysis with data ingestion",
    next_agent_suggestion="DataIngestionAgent",
    metadata={"reason": "initial_state"}
)
```

State Changes:

```
current_step = "ingestion"
agent_history.append({
    "agent": "PlannerAgent",
    "timestamp": "2026-07-31T22:00:01Z",
    "duration_seconds": 0.001,
    "result": AgentResult(...)
})
```

Next: DataIngestionAgent (via route_from_planner)

STEP 3: DataIngestionAgent

Component: DataIngestionAgent

Action: Load employees.csv

Process:

1. Validate file path exists
2. Detect format (CSV)
3. Try UTF-8 encoding
4. Parse CSV with pandas
5. Validate DataFrame not empty

Data Loaded:

```
raw_data = pd.DataFrame({
    'id': [1, 2, 3, ...],
    'name': ['Alice', 'Bob', 'Charlie', ...],
    'age': [28, 34, 45, ...],
    'salary': [75000, 82000, 95000, ...],
    'department': ['Engineering', 'Sales', 'Marketing', ...],
    ...
})
```

AgentResult:

```
AgentResult(
    decision=AgentDecision.CONTINUE,
    message="Successfully loaded 1000 rows from employees.csv",
    data_updates={
        "raw_data": raw_data,
        "file_metadata": {
            "file_size_bytes": 45000,
            "format": "csv",
            "encoding": "utf-8",
            "rows": 1000,
        }
    }
)
```

```

        "columns": 8
    },
    },
    metadata={"encoding_used": "utf-8"}
)

```

State Changes:

```

data = {
    "raw_data": raw_data,
    "file_metadata": {...}
}
steps_completed = ["DataIngestionAgent"]
agent_history.append({...})

```

Next: Planner Agent (via edge)

STEP 4: Planner Agent (Second Run)

Component: `PlannerAgent`

Action: Decide next agent after Ingestion

Decision Logic:

```

# DataIngestionAgent completed, DataProfilingAgent not completed
if "DataIngestionAgent" in steps and "DataProfilingAgent" not in steps:
    return self._after_ingestion(state)

```

AgentResult:

```

AgentResult(
    decision=AgentDecision.CONTINUE,
    message="Data loaded successfully, proceeding to profiling",
    next_agent_suggestion="DataProfilingAgent",
    metadata={"reason": "ingestion_complete"}
)

```

State Changes:

```

current_step = "profiling"
agent_history.append({...})

```

Next: DataProfilingAgent (via route_from_planner)

STEP 5: DataProfilingAgent

Component: DataProfilingAgent

Action: Analyze data structure

Process:

1. Analyze each column (name, type, missing values, unique values)
2. Classify semantic types (numeric, categorical, etc.)
3. Identify numeric and categorical columns
4. Detect potential keys
5. Generate initial insights

Column Analysis:

```
columns_info = {
    'id': {
        'name': 'id',
        'dtype': 'int64',
        'missing_pct': 0.0,
        'unique_count': 1000,
        'type': 'numeric',
        'is_potential_key': True
    },
    'name': {
        'name': 'name',
        'dtype': 'object',
        'missing_pct': 0.0,
        'unique_count': 950,
        'type': 'text'
    },
    'age': {
        'name': 'age',
        'dtype': 'int64',
        'missing_pct': 2.5,
        'unique_count': 45,
        'type': 'numeric'
    },
    'salary': {
        'name': 'salary',
        'dtype': 'int64',
        'missing_pct': 1.0,
        'unique_count': 850,
        'type': 'numeric'
    },
    'department': {
        'name': 'department',
        'dtype': 'object',
```

```

        'missing_pct': 0.0,
        'unique_count': 5,
        'type': 'categorical'
    },
    ...
}

```

Classification:

```

numeric_columns = ['id', 'age', 'salary', 'years_of_service', 'performance_score']
categorical_columns = ['department', 'remote_worker']
has_numeric_columns = True
has_categorical_columns = True

```

AgentResult:

```

AgentResult(
  decision=AgentDecision.CONTINUE,
  message="Profiled 8 columns: 5 numeric, 2 categorical, 1 text",
  data_updates={
    "profile": {
      "n_rows": 1000,
      "n_columns": 8,
      "columns": columns_info,
      "numeric_columns": numeric_columns,
      "categorical_columns": categorical_columns,
      "has_numeric_columns": True,
      "has_categorical_columns": True,
      "numeric_column_count": 5,
      "categorical_column_count": 2
    },
    "insights": [
      "Dataset contains 1000 rows and 8 columns",
      "5 numeric columns identified for statistical analysis",
      "2 categorical columns found (department, remote_worker)",
      "Missing values detected in age (2.5%) and salary (1.0%)",
      "id column appears to be a unique key"
    ]
  },
  metadata={"profiling_duration_seconds": 0.15}
)

```

State Changes:

```

data["profile"] = {...}
data["insights"] = [...]

```

```
steps_completed = ["DataIngestionAgent", "DataProfilingAgent"]
agent_history.append({...})
```

Next: Planner Agent (via edge)

STEP 6: Planner Agent (Third Run)

Component: `PlannerAgent`

Action: Decide next agent after Profiling

Decision Logic:

```
# DataProfilingAgent completed, StatisticalAnalysisAgent not completed
if "DataProfilingAgent" in steps and "StatisticalAnalysisAgent" not in steps:
    return self._after_profiling(state)
```

Logic in `_after_profiling`:

```
# Check if numeric columns exist
if profile.get("has_numeric_columns"):
    return AgentResult(
        decision=AgentDecision.CONTINUE,
        message="Numeric columns found, proceeding to statistical analysis",
        next_agent_suggestion="StatisticalAnalysisAgent"
    )
```

AgentResult:

```
AgentResult(
    decision=AgentDecision.CONTINUE,
    message="Numeric columns found, proceeding to statistical analysis",
    next_agent_suggestion="StatisticalAnalysisAgent",
    metadata={"reason": "numeric_columns_present"}
)
```

State Changes:

```
current_step = "statistics"
agent_history.append({...})
```

Next: StatisticalAnalysisAgent (via `route_from_planner`)

STEP 7: StatisticalAnalysisAgent

Component: StatisticalAnalysisAgent

Action: Perform statistical analysis

Process:

1. Get numeric columns from profile
2. Calculate descriptive statistics
3. Compute correlation matrix
4. Perform distribution tests
5. Detect outliers
6. Generate insights

Descriptive Statistics:

```
descriptive_stats = {  
    'age': {  
        'mean': 35.2,  
        'median': 34.0,  
        'std': 8.5,  
        'min': 22,  
        'max': 62,  
        'q25': 28.0,  
        'q75': 42.0  
    },  
    'salary': {  
        'mean': 78500.0,  
        'median': 76000.0,  
        'std': 15000.0,  
        'min': 45000,  
        'max': 125000,  
        'q25': 65000.0,  
        'q75': 90000.0  
    },  
    ...  
}
```

Correlation Matrix:

```
correlation_matrix = {  
    'age': {'age': 1.0, 'salary': 0.65, 'years_of_service': 0.82, ...},  
    'salary': {'age': 0.65, 'salary': 1.0, 'years_of_service': 0.78, ...},  
    ...  
}
```

Distribution Tests:

```
distribution_tests = {
    'age': {
        'shapiro_stat': 0.98,
        'shapiro_p': 0.02,
        'is_normal': False,
        'skewness': 0.35,
        'kurtosis': -0.42
    },
    ...
}
```

Outliers:

```
outliers = {
    'age': [62, 61, 60], # High outliers
    'salary': [125000, 120000, 118000] # High outliers
}
```

AgentResult:

```
AgentResult(
    decision=AgentDecision.CONTINUE,
    message="Statistical analysis complete for 5 numeric columns",
    data_updates={
        "statistics": {
            "descriptive_stats": descriptive_stats,
            "correlation_matrix": correlation_matrix,
            "distribution_tests": distribution_tests,
            "outliers": outliers
        },
        "insights": [
            "Strong positive correlation (0.82) between age and years_of_service",
            "Salary moderately correlated with age (0.65) and years_of_service",
            "(0.78)",
            "Age distribution is slightly right-skewed (skewness: 0.35)",
            "3 outliers detected in age and salary columns",
            "Mean salary: $78,500, median: $76,000"
        ]
    },
    metadata={"statistics_duration_seconds": 0.25}
)
```

State Changes:

```
data["statistics"] = {...}
data["insights"].extend([...])
```

```
steps_completed = ["DataIngestionAgent", "DataProfilingAgent",  
"StatisticalAnalysisAgent"]  
agent_history.append({...})
```

Next: Planner Agent (via edge)

STEP 8: Planner Agent (Fourth Run)

Component: `PlannerAgent`

Action: Decide next agent after Statistics

Decision Logic:

```
# StatisticalAnalysisAgent completed, VisualizationAgent not completed  
if "StatisticalAnalysisAgent" in steps and "VisualizationAgent" not in steps:  
    return self._after_statistics(state)
```

AgentResult:

```
AgentResult(  
    decision=AgentDecision.CONTINUE,  
    message="Statistical analysis complete, proceeding to visualization",  
    next_agent_suggestion="VisualizationAgent",  
    metadata={"reason": "statistics_complete"}  
)
```

State Changes:

```
current_step = "visualization"  
agent_history.append({...})
```

Next: VisualizationAgent (via route_from_planner)

STEP 9: VisualizationAgent

Component: `VisualizationAgent`

Action: Generate visualizations

Process:

1. Get numeric and categorical columns from profile
2. Generate histograms for numeric columns

- 3. Generate box plots for numeric columns
- 4. Generate scatter plots for numeric pairs
- 5. Generate bar charts for categorical columns
- 6. Generate correlation heatmap
- 7. Save all as HTML files

Visualizations Generated:

Type	Columns	File
Histogram	age	distribution_age.html
Histogram	salary	distribution_salary.html
Box Plot	age	boxplot_age.html
Box Plot	salary	boxplot_salary.html
Scatter	age vs salary	scatter_age_salary.html
Scatter	age vs years_of_service	scatter_age_years_of_service.html
Bar Chart	department	bar_department.html
Bar Chart	remote_worker	bar_remote_worker.html
Heatmap	All numeric	correlation_heatmap.html

Visualization Metadata:

```
visualizations = [  
  {  
    "type": "histogram",  
    "column": "age",  
    "file": "distribution_age.html",  
    "title": "Distribution of Age"  
  },  
  {  
    "type": "boxplot",  
    "column": "salary",  
    "file": "boxplot_salary.html",  
    "title": "Salary Distribution with Outliers"  
  },  
  {  
    "type": "scatter",  
    "x_column": "age",  
    "y_column": "salary",  
    "file": "scatter_age_salary.html",  
    "title": "Age vs Salary Correlation"  
  },  
  ...  
]
```

Visualization Insights:

```
insights = [  
    "Age distribution shows slight right skew, mean (35.2) > median (34)",  
    "Salary distribution is right-skewed with outliers above $110K",  
    "Strong positive correlation visible between age and salary",  
    "Engineering department has highest salary distribution",  
    "Remote workers show slightly higher median salary"  
]
```

AgentResult:

```
AgentResult(  
    decision=AgentDecision.CONTINUE,  
    message="Generated 25 visualizations: histograms, box plots, scatter plots,  
bar charts, heatmap",  
    data_updates={  
        "visualizations": visualizations,  
        "insights": insights  
    },  
    metadata={  
        "visualization_count": 25,  
        "visualization_duration_seconds": 1.5  
    }  
)
```

State Changes:

```
data["visualizations"] = [...]  
data["insights"].extend([...])  
steps_completed = ["DataIngestionAgent", "DataProfilingAgent",  
"StatisticalAnalysisAgent", "VisualizationAgent"]  
agent_history.append({...})
```

Next: Planner Agent (via edge)

STEP 10: Planner Agent (Fifth Run)

Component: `PlannerAgent`

Action: Decide next agent after Visualization

Decision Logic:

```
# VisualizationAgent completed, EvaluatorAgent not completed
if "VisualizationAgent" in steps and "EvaluatorAgent" not in steps:
    return self._after_visualization(state)
```

AgentResult:

```
AgentResult(
    decision=AgentDecision.CONTINUE,
    message="Visualizations generated, proceeding to evaluation",
    next_agent_suggestion="EvaluatorAgent",
    metadata={"reason": "visualization_complete"}
)
```

State Changes:

```
current_step = "evaluator"
agent_history.append({...})
```

Next: EvaluatorAgent (via route_from_planner)

STEP 11: EvaluatorAgent

Component: EvaluatorAgent

Action: Validate analysis results

Process:

1. Check if data was loaded (`_check_has_data`)
2. Check if insights were generated (`_check_has_insights`)
3. Check data quality (`_check_data_quality`)
4. Check analysis depth (`_check_analysis_depth`)

Validation Checks:

```
checks = {
    "has_data": True, # raw_data exists and has 1000 rows
    "has_insights": True, # insights list has 15+ items
    "data_quality": True, # missing values < 50%
    "sufficient_depth": True # multiple analysis steps completed
}
```

Check Details:

Check	Result	Details
has_data	✓ PASS	raw_data has 1000 rows, 8 columns
has_insights	✓ PASS	15 insights generated
data_quality	✓ PASS	Max missing % is 2.5% (age)
sufficient_depth	✓ PASS	4 agents completed successfully

AgentResult:

```
AgentResult(  
  decision=AgentDecision.CONTINUE,  
  message="All validation checks passed",  
  data_updates={  
    "validation_status": "passed",  
    "validation_errors": []  
  },  
  metadata={  
    "validation_results": checks,  
    "all_checks_passed": True  
  }  
)
```

State Changes:

```
data["validation_status"] = "passed"  
data["validation_errors"] = []  
steps_completed = ["DataIngestionAgent", "DataProfilingAgent",  
  "StatisticalAnalysisAgent", "VisualizationAgent", "EvaluatorAgent"]  
agent_history.append({...})
```

Next: Planner Agent (via edge)

STEP 12: Planner Agent (Sixth Run)

Component: PlannerAgent

Action: Decide next agent after Evaluator

Decision Logic:

```
# EvaluatorAgent completed, ReportingAgent not completed  
if "EvaluatorAgent" in steps and "ReportingAgent" not in steps:  
    return self._after_evaluator(state)
```

Logic in `_after_evaluator`:

```
# Check validation status
if state.get("validation_status") == "passed":
    return AgentResult(
        decision=AgentDecision.CONTINUE,
        message="Validation passed, proceeding to report generation",
        next_agent_suggestion="ReportingAgent"
    )
else:
    # Handle validation failure...
```

AgentResult:

```
AgentResult(
    decision=AgentDecision.CONTINUE,
    message="Validation passed, proceeding to report generation",
    next_agent_suggestion="ReportingAgent",
    metadata={"reason": "validation_passed"}
)
```

State Changes:

```
current_step = "reporting"
agent_history.append({...})
```

Next: ReportingAgent (via `route_from_planner`)

STEP 13: ReportingAgent

Component: ReportingAgent

Action: Generate comprehensive report

Process:

1. Gather all analysis results
2. Generate HTML report with embedded visualizations
3. Generate JSON report for automation
4. Save both files

Report Metadata:

```
report_metadata = {
    "generated_at": "2026-07-31T22:00:05Z",
```

```
"dataset_path": "datasets/employees.csv",
"total_rows": 1000,
"total_columns": 8,
"execution_duration_seconds": 5.2,
"agents_executed": 6,
"visualizations_generated": 25,
"insights_generated": 15
}
```

HTML Report Contents:

- Header with metadata
- Data Profile section
- Statistics section
- Visualizations section (embedded)
- Insights section
- Recommendations section

JSON Report Contents:

```
{
  "metadata": {...},
  "profile": {...},
  "statistics": {...},
  "visualizations": [...],
  "insights": [...],
  "recommendations": [...]
}
```

AgentResult:

```
AgentResult(
  decision=AgentDecision.COMPLETE,
  message="Report generated successfully",
  data_updates={
    "report_path": "./output/employees/report.html",
    "report_json_path": "./output/employees/report.json",
    "report_metadata": report_metadata
  },
  metadata={
    "html_report_size_bytes": 125000,
    "json_report_size_bytes": 45000,
    "report_duration_seconds": 0.8
  }
)
```

State Changes:

```

data["report_path"] = "./output/employees/report.html"
data["report_json_path"] = "./output/employees/report.json"
data["report_metadata"] = {...}
end_time = "2026-07-31T22:00:05Z"
steps_completed = ["DataIngestionAgent", "DataProfilingAgent",
"StatisticalAnalysisAgent", "VisualizationAgent", "EvaluatorAgent",
"ReportingAgent"]
agent_history.append({...})

```

Next: END (via edge)

STEP 14: Complete

Component: CLI (`cli.py`)

Action: Display completion message

Final State:

```

GraphState(
    input_dataset_path="datasets/employees.csv",
    input_query=None,
    output_dir="./output/employees",
    execution_id="exec_1234567890",
    current_step="reporting",
    steps_completed=[
        "DataIngestionAgent",
        "DataProfilingAgent",
        "StatisticalAnalysisAgent",
        "VisualizationAgent",
        "EvaluatorAgent",
        "ReportingAgent"
    ],
    agent_history=[...], # 6 entries
    validation_status="passed",
    validation_errors=[],
    retry_count=0,
    max_retries=3,
    logs=[...],
    metrics={...},
    start_time="2026-07-31T22:00:00Z",
    end_time="2026-07-31T22:00:05Z",
    data={
        "raw_data": DataFrame(...),
        "file_metadata": {...},
        "profile": {...},
        "statistics": {...},
        "visualizations": [...],
        "insights": [...],
        "validation_status": "passed",
    }
)

```

```
        "validation_errors": [],
        "report_path": "./output/employees/report.html",
        "report_json_path": "./output/employees/report.json",
        "report_metadata": {...}
    }
)
```

Output Displayed:

✓ Dataset Loaded

✓ Profiling Complete

✓ Statistics Complete

✓ Visualizations Generated

✓ Report Generated

✓ Workflow Finished

✓ Execution duration: 5.20s

 Analysis complete! Report saved to: ./output/employees/report.html

✓ Logs saved to: ./output/employees/execution_workflow.log

Exit Code: 0 (success)

Execution Summary

Metric	Value
Total Duration	5.2 seconds
Agents Executed	6
Planner Runs	6
Visualizations Generated	25
Insights Generated	15
Validation Status	Passed
Retry Count	0
Exit Code	0

PHASE 7: Code Review

Best Engineering Decisions

1. Single Source of Truth (GraphState)

Decision: Use a single **GraphState** object that flows through all agents.

Why it's excellent:

- Eliminates state synchronization issues
- Makes data flow explicit and traceable
- Simplifies debugging with complete execution history
- Enables easy replay and testing

Where it's implemented:

- `dataforge/core/state.py` - State model definition
 - `dataforge/graph/workflow.py` - State flows through graph
 - All agents receive and return state
-

2. Immutable State Updates

Decision: State updates create new state objects rather than mutating existing ones.

Why it's excellent:

- Prevents accidental side effects
- Makes state changes explicit and traceable
- Enables functional programming patterns
- Simplifies concurrent execution (future-proof)

Where it's implemented:

- `dataforge/core/state.py` - `set()` method returns new state
 - `dataforge/agents/base.py` - `execute_with_logging()` creates new state
-

3. Agent Abstraction

Decision: All agents inherit from a common `Agent` base class with standardized interface.

Why it's excellent:

- Enforces consistent behavior across all agents
- Simplifies adding new agents
- Enables automatic logging and error handling
- Makes testing easier with mockable interface

Where it's implemented:

- `dataforge/agents/base.py` - Base agent class
 - All agent files inherit from this base
-

4. Centralized Decision Making (Planner)

Decision: Single `PlannerAgent` makes all routing decisions.

Why it's excellent:

- Separates orchestration logic from business logic
- Makes workflow changes easy (modify planner only)
- Enables dynamic routing based on state
- Simplifies debugging of flow control

Where it's implemented:

- `dataforge/agents/planner.py` - Planner agent
 - `dataforge/graph/workflow.py` - Routing function
-

5. LLM Provider Abstraction

Decision: Abstract LLM providers behind `LLMProvider` interface.

Why it's excellent:

- Eliminates vendor lock-in
- Makes adding new providers trivial
- Enables provider switching without code changes
- Simplifies testing with mock providers

Where it's implemented:

- `dataforge/core/llm.py` - Provider interface
 - `dataforge/infrastructure/llm_providers/` - Implementations
-

6. Quality Gates (Evaluator)

Decision: Separate `EvaluatorAgent` validates results before reporting.

Why it's excellent:

- Ensures output quality before presentation
- Enables automated quality control
- Provides clear feedback on failures
- Supports retry logic for transient issues

Where it's implemented:

- `dataforge/agents/evaluator.py` - Evaluator agent
-

7. Structured Logging

Decision: `StructuredLogger` provides consistent, queryable logging.

Why it's excellent:

- Enables log aggregation and analysis
- Provides execution audit trail
- Simplifies debugging with structured data
- Supports log export for external tools

Where it's implemented:

- [dataforge/core/logger.py](#) - Logger implementation
 - All agents use logger for consistent output
-

Clean Architecture Examples

1. Layer Separation

Architecture:

```
Presentation Layer (CLI)
    ↓
Graph Orchestration Layer (Workflow)
    ↓
Agent Layer (Business Logic)
    ↓
Core Layer (Domain Models)
    ↓
Infrastructure Layer (External Services)
```

Why it's clean:

- Each layer has single responsibility
- Dependencies flow inward (no circular dependencies)
- Easy to test each layer independently
- Can swap implementations without affecting other layers

Where it's implemented:

- [dataforge/presentation/](#) - CLI
 - [dataforge/graph/](#) - Workflow
 - [dataforge/agents/](#) - Business logic
 - [dataforge/core/](#) - Domain models
 - [dataforge/infrastructure/](#) - External services
-

2. Dependency Inversion

Decision: High-level modules depend on abstractions, not concrete implementations.

Example: LLM providers

- Agents depend on [LLMProvider](#) interface

- Concrete implementations (OpenAI, Anthropic) implement interface
- Can add new providers without modifying agents

Where it's implemented:

- [dataforge/core/llm.py](#) - Abstract interface
 - [dataforge/infrastructure/llm_providers/](#) - Concrete implementations
-

3. Interface Segregation

Decision: Small, focused interfaces rather than large, monolithic ones.

**Example:*